

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....	2
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	3
ВВЕДЕНИЕ.....	4
1 РАЗРАБОТКА КОНЦЕПТУАЛЬНОЙ МОДЕЛИ КЛИЕНТ-СЕРВЕРНОЙ СИСТЕМЫ.....	6
1.1 Идентификация предметной области автоматизации	6
1.2 Обзор и анализ аналогичных клиент-серверной системы.....	8
1.3 Выбор методологии и технологии концептуального моделирования клиент- серверной системы.....	10
1.4 Разработка и анализ модели бизнес-процесса (AS-IS).....	11
1.5 Разработка функциональной модели (TO-BE).....	13
1.6 Разработка требований к клиент-серверной системе.....	15
1.7 Выводы и результаты к разделу	20
2 РАЗРАБОТКА ЛОГИЧЕСКОЙ МОДЕЛИ КЛИЕНТ-СЕРВЕРНОЙ СИСТЕМЫ.....	22
2.1 Выбор методологии и технологии логического моделирования клиент-серверной системы.....	22
2.2 Разработка диаграмм логической модели клиент-серверной системы ..	22
2.3 Разработка логической модели данных клиент-серверной системы.....	27
2.4 Разработка архитектуры клиент-серверной системы с использованием паттерна, целесообразность которого обоснована	28
2.5 Выводы и результаты к разделу	30
ЗАКЛЮЧЕНИЕ	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	Ошибка! Закладка не определена.

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете применяют следующие термины с соответствующими определениями:

Платёжный шлюз	— программно-аппаратный комплекс, обеспечивающий передачу информации о платеже между интернет-магазином, платёжной системой и банком-эквайером с гарантией безопасности транзакций
Микросервис	— независимый модуль системы, выполняющий определённую бизнес-логику
Агрегатор платежей	— сервис, объединяющий различные способы оплаты и банки через единую точку интеграции, позволяющий мерчантам принимать платежи без прямого подключения к каждому поставщику услуг
Антифрод	— система выявления и предотвращения мошеннических транзакций на основе анализа поведения плательщика, риск-скоринга и проверки подозрительных паттернов

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете применяются следующие сокращения и обозначения:

СУБД	—	система управления базами данных
API	—	Application Programming Interface
BPMN	—	Business Process Model and Notation
PCI DSS	—	Payment Card Industry Data Security Standard
PSP	—	Payment Service Provider
p95	—	95-й процентиль
SQL	—	Structured Query Language
UML	—	Unified Modeling Language

ВВЕДЕНИЕ

В современной цифровой экономике стремительно растет объем и сложность онлайн-платежей, что предъявляет новые требования к надежности, безопасности и универсальности платежной инфраструктуры. Предприятиям, особенно малым и средним, зачастую экономически нецелесообразно и технически сложно интегрироваться с каждым банком или платежной системой отдельно. Традиционные подходы, предполагающие прямые интеграции, характеризуются высокими затратами на разработку и поддержку, сложностью обеспечения соответствия стандартам безопасности, отсутствием единой точки для аналитики и управления транзакциями, а также низкой отказоустойчивостью. Это приводит к увеличению времени выхода на рынок, рискам финансовых потерь из-за мошенничества или сбоев и, как следствие, к снижению конкурентоспособности бизнеса.

Актуальность разработки специализированного платежного шлюза-агрегатора обусловлена необходимостью создания универсального, безопасного и масштабируемого посредника между торговыми точками (мерчантами) и множеством финансовых провайдеров (банками, платежными системами, эквайерами). Современные реалии финтеха требуют обеспечения сквозной безопасности данных, высокой доступности системы, эффективной маршрутизации платежей для оптимизации комиссий и успешности операций, а также предоставления мерчанту удобного инструментария для управления платежами и получения консолидированной отчетности.

Целью исследования является проектирование клиент-серверной системы платежного шлюза-агрегатора для обработки онлайн-платежей.

Для достижения данной цели были поставлены следующие задачи:

- разработка концептуальной модели клиент-серверной системы;
- разработка логической модели клиент-серверной системы.

Объектом исследования является клиент-серверная система обработки финансовых транзакций, выступающая в роли унифицированного платежного интерфейса для мерчантов.

Предметом исследования являются архитектура, функциональность, модель данных и нефункциональные характеристики проектируемой системы, включающие модули приема платежей, маршрутизации, взаимодействия с внешними API, безопасности и администрирования.

Методологическая основа исследования включает принципы проектирования высоконагруженных отказоустойчивых систем, методологии объектно-ориентированного анализа и проектирования (UML) [1], CASE-технологии [2], а также использование нотаций C4 [3] и ArchiMate [4] для визуализации архитектурных решений, BPMN [5] для моделирования платежных процессов.

Структура работы состоит из двух основных глав. В первой главе разрабатывается концептуальная модель системы: проводится анализ предметной области и требований, выявляются ключевые участники и процессы. Во второй главе проектируется логическая модель, включающая архитектуру, схему взаимодействия компонентов и детализацию ключевых модулей, обеспечивающих безопасную и надежную обработку транзакций.

1 РАЗРАБОТКА КОНЦЕПТУАЛЬНОЙ МОДЕЛИ КЛЕНТ-СЕРВЕРНОЙ СИСТЕМЫ

1.1 Идентификация предметной области автоматизации

Объектом исследования выступает комплексный процесс обработки онлайн-платежей, который включает приём платёжного запроса от клиента, безопасную обработку данных, маршрутизацию транзакции к финансовому провайдеру, подтверждение результата и взаиморасчёты между участниками (мерчантом, агрегатором и банком).

Объектом автоматизации является комплекс операционных процессов, обеспечивающих безопасность, надёжность и универсальность проведения финансовых транзакций в интернет-среде. Это включает валидацию платёжных данных, безопасное следование стандарту PCI DSS, маршрутизацию платежа к оптимальному провайдеру, обработку асинхронных ответов, защиту от мошенничества и корректное проведение взаиморасчётов. От эффективности данных процессов напрямую зависят ключевые бизнес-показатели: конверсия успешных платежей, уровень финансовых потерь, операционная надёжность и доверие со стороны мерчантов и конечных плательщиков.

В работе системы участвуют три основные категории пользователей (акторов):

- Плательщик (Клиент) – конечный пользователь, который инициирует оплату товара или услуги на стороне мерчанта, предоставляет данные платёжного средства и подтверждает транзакцию;
- Мерчант (Торговая точка) – юридическое лицо или индивидуальный предприниматель, интегрирующий платёжный шлюз на свой сайт или в приложение для приёма оплаты от клиентов. Управляет настройками платежей, просматривает историю транзакций и получает отчёты;
- Администратор платёжного шлюза – сотрудник компании-агрегатора, ответственный за общее управление системой: подключение

новых мерчантов и платёжных провайдеров, конфигурацию маршрутизации, мониторинг транзакций, разрешение инцидентов и формирование финансовой отчётности.

Основные бизнес-функции, подлежащие автоматизации, включают: приём и валидацию платёжных запросов, безопасное хранение и токенизацию данных платёжных средств, интеллектуальную маршрутизацию транзакций к банкам-эквайерам и платёжным системам, обработку 3-D Secure аутентификации, приём и обработку асинхронных уведомлений от провайдеров, автоматизированное выявление подозрительных операций, проведение возвратов, а также генерацию консолидированной финансовой отчётности для мерчантов и внутреннего учёта.

Позиция проектируемой системы в экосистеме взаимодействия между участниками платежей отражена на диаграмме контекста C4, представленной на рисунке 1.1.

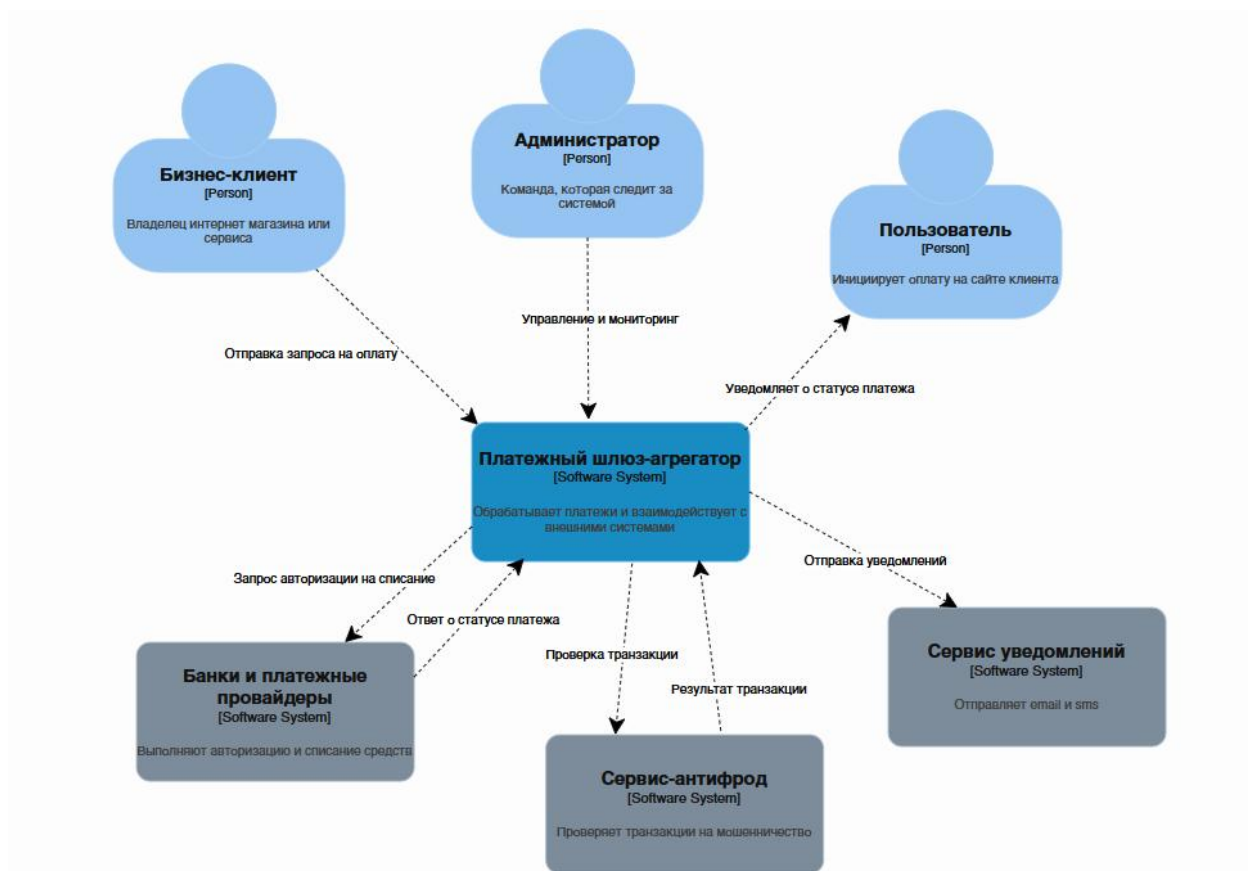


Рисунок 1.1 – Диаграмма C4 System Context

Система взаимодействует с набором внешних сервисов, обеспечивающих критически важные функции:

- интернет-магазин (мерчант) – для приема платежного запроса от конечного клиента и отображения статуса оплаты;
- платежные провайдеры (банки, платежные системы) – для непосредственного проведения финансовых транзакций;
- сервис антифрода – для анализа транзакций на мошеннические действия и отправки результата проверки.

1.2 Обзор и анализ аналогичных клиент-серверной системы

На рынке представлены готовые коммерческие решения, выполняющие роль платежного шлюза-агрегатора. Однако для целей архитектурного проектирования целесообразно провести сравнительный анализ ключевых функциональных возможностей и технологических подходов, характерных для данного класса систем, таких как: поддержка множества платежных методов, модель маршрутизации транзакций, реализация.

Сравнительный анализ данных характеристик представлен в таблице 1.1.

Таблица 1.1 – Сравнительный анализ платежных шлюзов-агрегаторов

Критерий	CloudPayments[6]	ЮKassa[7]	PayU[8]
Основная специализация	Платежный сервис с собственным эквайрингом, ориентированный на рынок РФ и СНГ.	Крупнейший агрегатор платежей с максимальным охватом методов оплаты на рынке РФ.	Глобальный платежный агрегатор для emerging markets с акцентом на локальные способы оплаты.
Поддержка платежных методов	Карты, СБП, Apple/Google Pay, часть электронных кошельков.	Карты, СБП, электронные кошельки, криптовалюты, рассрочка.	Карты и широкий спектр локальных способов оплаты в каждой стране присутствия.
Архитектурная модель и интеграция	SaaS-платформа с предоставлением API, виджетов и SDK для разработчиков.	Единая SaaS-платформа с типовыми сценариями интеграции.	Глобальная стандартизированная платформа с единым API для всех регионов.

Управление и кастомизация	Базовая настройка правил через личный кабинет. Глубокая кастомизация ограничена.	Управление через личный кабинет. Возможности изменения встроеной логики маршрутизации и обработки отсутствуют.	Стандартизированное управление. Адаптация платформы под уникальные требования заказчика не предусмотрена.
Модель безопасности (PCI DSS)	Ответственность за соответствие разделена между сервисом и мерчантом.	Мерчант работает в зоне ответственности агрегатора, что упрощает процесс сертификации.	Глобальная сертификация платформы. Мерчант интегрируется как часть сертифицированной экосистемы.

Анализ представленных на рынке решений показывает, что ни одно из них в полной мере не отвечает комплексу требований, предъявляемых к проектируемой системе. Ключевой задачей является создание не просто платежного интерфейса, а управляемой платформы, где каждый компонент — от маршрутизации до протоколов безопасности — может быть адаптирован под конкретные бизнес-процессы.

Адаптация готовых коммерческих продуктов сопряжена со следующими системными ограничениями:

- Фиксированная архитектура коммерческих платформ не предусматривает изменения базовой логики, такой как алгоритмы выбора платежного провайдера или обработки ошибочных сценариев. Это делает их непригодными для реализации специализированных требований к потоку транзакций;

- Стандартизированные модели безопасности данных, предлагаемые агрегаторами, часто несовместимы с необходимостью полного внутреннего контроля над криптографией и хранением информации, что критично для соблюдения корпоративных или отраслевых стандартов, отличных от типовой схемы PCI DSS;

- Возможности интеграции через предоставляемые API и вебхуки рассчитаны на усредненные сценарии использования. Они не поддерживают реализацию сложной кастомной логики, например, для динамического

биллинга, многоуровневой обработки уведомлений или глубокой синхронизации с внутренними системами учета.

Следовательно, оптимальным путем является самостоятельное проектирование системы. Такой подход позволяет разработать целостное решение, архитектура которого изначально ориентирована на требуемую гибкость, полную управляемость платежными процессами и бесшовную интеграцию в существующую ИТ-инфраструктуру, что обеспечивает долгосрочную технологическую независимость и эффективность.

1.3 Выбор методологии и технологии концептуального моделирования клиент-серверной системы

Для концептуального моделирования системы был проведён анализ современных методологий, результаты которого представлены в таблице 1.2.

Таблица 1.2 – Сравнительный анализ методологий

Критерий	IDEF0	DFD	UML	BPMN
Детализация модели	Функциональные блоки и их взаимосвязи	Потоки данных между процессами и хранилищами	Объекты, состояния, взаимодействие компонентов	Операции и этапы бизнес-процессов
Наглядность диаграмм	Условная, структурная	Четкая, акцент на движении данных	Высокая за счет разнообразия типов диаграмм	Максимальная, интуитивная для бизнес-аналитиков
Возможность выполнения	Отсутствует	Отсутствует	Частичная	Полная
Связь с разработкой ПО	Слабая	Умеренная	Прямая и тесная	Умеренная или тесная

Для проектирования системы выбрана комбинация методологий BPMN и UML. BPMN применяется для моделирования бизнес-процессов, так как обеспечивает их наглядное и исполняемое описание. UML используется для объектно-ориентированного проектирования программной системы, включая диаграммы вариантов использования, классов и последовательностей.

Таким образом, комбинация BPMN и UML позволяет разделить описание бизнес-логики и технической реализации. Это повышает качество проектирования, особенно в контексте сложной, распределенной клиент-

серверной архитектуры. Для реализации моделей будут использованы современные CASE-средства, поддерживающие обе нотации.

1.4 Разработка и анализ модели бизнес-процесса (AS-IS)

На основе методологии BPMN была разработана модель текущего (AS IS) бизнес-процесса обработки онлайн-платежа, которая отражает существующую неэффективную процедуру. Процесс включает взаимодействие плательщика (клиента), мерчанта (торговой точки), сотрудника мерчанта и банка-эквайера. Диаграмма представлена на рисунке 1.2.

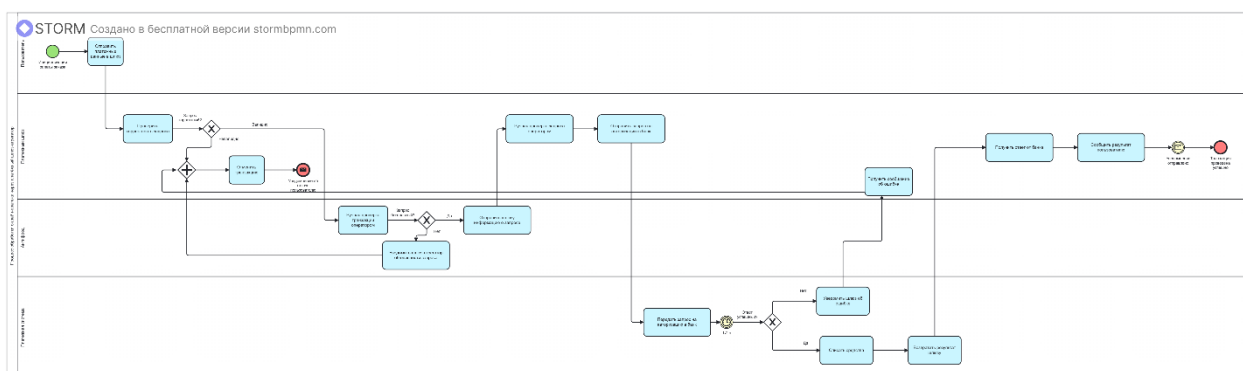


Рисунок 1.2 – Диаграмма текущего бизнес-процесса обработки онлайн-платежа в методологии BPMN

Процесс обработки онлайн-платежа начинается с инициации оплаты клиентом в пользовательском интерфейсе. Клиент вводит платёжные данные, которые передаются в платежный шлюз-агрегатор для дальнейшей обработки. На следующем этапе выполняется проверка корректности запроса, направленная на выявление ошибок ввода и несоответствий формата данных. В случае обнаружения некорректного запроса транзакция отклоняется, а клиенту направляется уведомление об ошибке.

При успешном прохождении первичной проверки транзакция поступает на этап проверки безопасности, который в текущей реализации осуществляется преимущественно вручную. Оператор платежного шлюза анализирует параметры транзакции и принимает решение о возможности её дальнейшей обработки. При выявлении признаков подозрительной

активности транзакция отклоняется, а информация о потенциально небезопасном запросе передается в систему учёта инцидентов.

Если по результатам ручной проверки транзакция признаётся допустимой, запрос на авторизацию передаётся в банк-эмитент. Банк обрабатывает запрос и возвращает результат авторизации. В случае отказа со стороны банка клиент уведомляется о неуспешном платеже. При положительном ответе выполняется списание средств, после чего результат транзакции передаётся платежному шлюзу-агрегатору и сообщается клиенту. Процесс завершается фиксацией статуса платежа в системе.

Выявленные недостатки процесса:

- наличие ручных этапов проверки транзакций: значительная часть операций, связанных с анализом безопасности платежа, выполняется оператором вручную, что увеличивает общее время обработки транзакции и повышает риск человеческого фактора;
- отсутствие автоматизированного механизма комплексной оценки безопасности: система не использует централизованную автоматическую проверку транзакций, из-за чего возможны как необоснованные отказы, так и пропуск потенциально рискованных операций;
- задержки при обработке платежей: дополнительная ручная проверка перед отправкой запроса в банк приводит к увеличению времени ожидания результата для клиента и снижает удобство использования системы;
- ограниченная обратная связь для клиента: при отказе в проведении транзакции клиент получает минимальную информацию о причине ошибки, что затрудняет повторную попытку оплаты и снижает уровень доверия к системе.

Проведённый анализ показывает, что текущий бизнес-процесс обработки онлайн-платежей характеризуется недостаточным уровнем автоматизации, повышенной нагрузкой на операторов и сниженной скоростью выполнения транзакций. Несмотря на работоспособность процесса, его реализация остаётся ресурсоёмкой и подверженной задержкам. Переход к

более автоматизированной архитектуре с использованием интеллектуальных механизмов проверки и оптимизации взаимодействия между компонентами клиент-серверной системы позволит повысить эффективность обработки платежей и улучшить пользовательский опыт.

1.5 Разработка функциональной модели (ТО-ВЕ)

На основе проведённого анализа предметной области разработана функциональная модель целевой системы платёжного шлюза-агрегатора. Модель, представленная на рисунках 1.3 и 1.4, построена с использованием нотации С4 и отражает целевую архитектуру системы «КАК ДОЛЖНО БЫТЬ» (ТО-ВЕ).

Диаграмма системного ландшафта (рис. 1.3) отражает экосистему взаимодействующих информационных систем, участвующих в процессе обработки онлайн-платежей. В рамках данной диаграммы отсутствует выделенный центральный компонент, а все системы рассматриваются как равноправные элементы общей архитектуры. Взаимодействие осуществляется между пользователем, системой мерчанта (интернет-магазином), платёжным шлюзом-агрегатором, банковскими системами, антифрод-сервисом и сервисом уведомлений.

Пользователь инициирует оплату через систему мерчанта, которая, в свою очередь, передаёт платёжные данные в платёжный шлюз. Платёжный шлюз взаимодействует с банковскими системами для выполнения авторизации и списания средств, а также с антифрод-сервисом для проверки транзакций. Сервис уведомлений обеспечивает информирование пользователей о статусе платежа. Таким образом, диаграмма демонстрирует общую структуру взаимодействия систем без детализации их внутренней реализации.

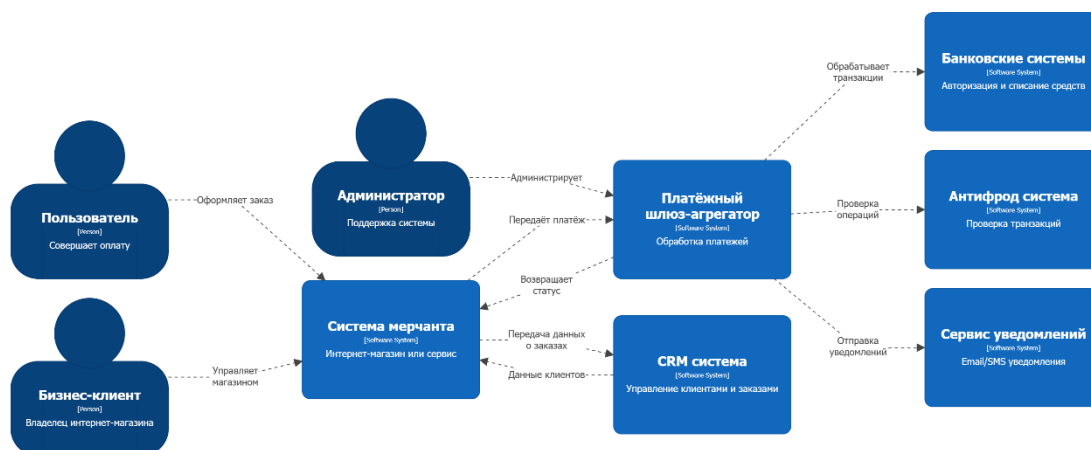


Рисунок 1.3 – Диаграмма C4 System LandScape Diagram

Диаграмма контейнеров (рис. 1.4) детализирует внутреннюю структуру платёжного шлюза-агрегатора. Архитектура системы построена по принципам микросервисного подхода с выделением ключевых функциональных компонентов и единой точки входа.

Входящей точкой в систему является API Gateway, который выполняет функции маршрутизации запросов, управления доступом, а также распределения нагрузки между внутренними сервисами. Основная бизнес-логика реализована в виде набора специализированных компонентов: оркестратор платежей управляет жизненным циклом транзакций и координирует взаимодействие между сервисами; платёжный движок отвечает за обработку операций оплаты; модуль маршрутизации определяет банк или платёжного провайдера для выполнения транзакции; антифрод-сервис осуществляет проверку операций на предмет мошенничества; сервис уведомлений отвечает за отправку сообщений пользователям.

Для хранения данных используется реляционная база данных PostgreSQL, обеспечивающая сохранность информации о платежах, транзакциях и пользователях. Для повышения производительности и реализации асинхронного взаимодействия применяется кэширование и

брокеры сообщений (например, Redis и Kafka), что позволяет эффективно обрабатывать большое количество параллельных операций.

Система интегрируется с внешними сервисами, включая банковские системы и платёжные сети для проведения транзакций, а также сервисы отправки уведомлений (Email/SMS). Взаимодействие между компонентами осуществляется посредством API и событийных механизмов, что обеспечивает масштабируемость и отказоустойчивость системы.

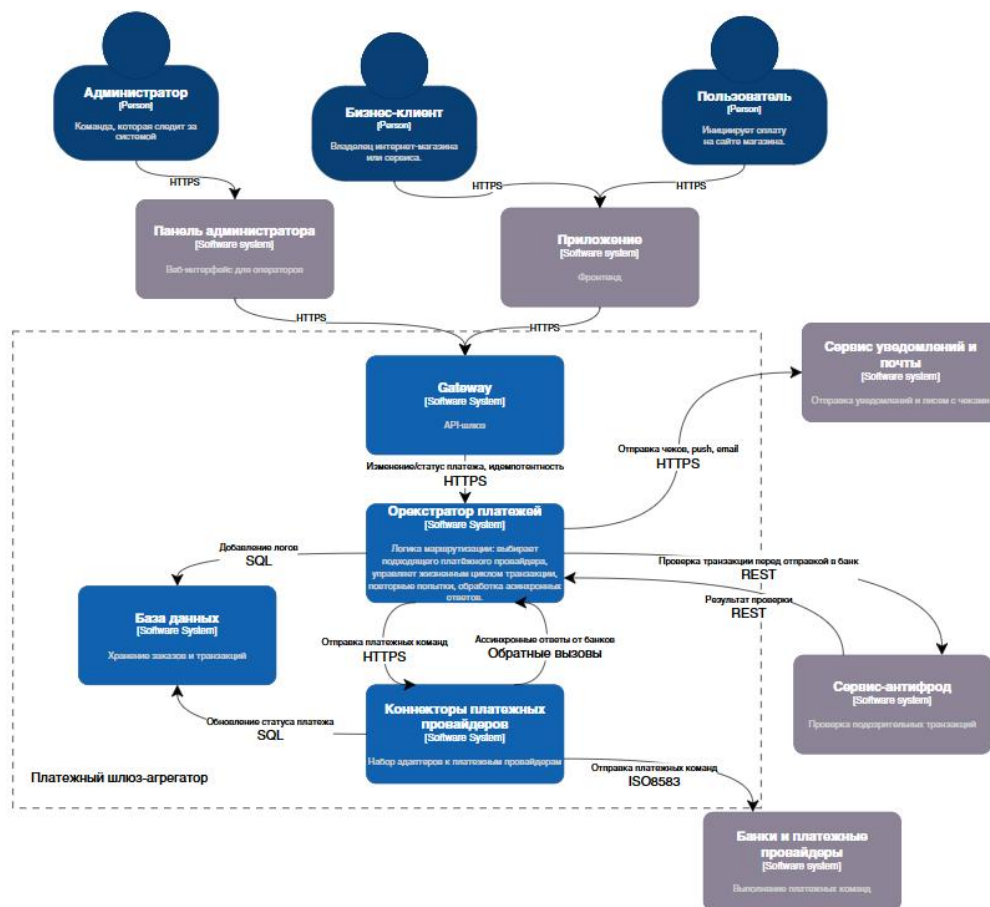


Рисунок 1.4 – Диаграмма C4 Container Diagram

1.6 Разработка требований к клиент-серверной системе

1.6.1 Цели и назначение создания системы

Проектируемая клиент-серверная система предназначена для автоматизации процессов обработки онлайн-платежей. Основной целью её создания является обеспечение надёжности, безопасности и высокой скорости проведения платёжных операций между пользователями, мерчантами и банковскими системами. Это достигается за счёт внедрения механизмов

маршрутизации транзакций, антифрод-проверок, обработки платежей и системы уведомлений.

1.6.2 Требования к функциям системы

Система должна обеспечивать полный цикл обработки онлайн-платежа: от инициации оплаты пользователем до получения результата транзакции и уведомления участников процесса. Функционал для пользователей включает возможность безопасной оплаты товаров и услуг, а также получение информации о статусе платежа. Для бизнес-клиентов система предоставляет инструменты для интеграции с платёжным шлюзом, отправки платёжных запросов и получения результатов операций. Администраторы получают доступ к интерфейсам мониторинга и управления системой, включая контроль транзакций, анализ ошибок и управление интеграциями с внешними сервисами. Система также обеспечивает взаимодействие с банковскими и антифрод-сервисами для проверки и выполнения транзакций, а также с сервисами уведомлений для информирования пользователей о статусе платежей.

1.6.3 Требования к производительности и надежности системы

Для обеспечения стабильной работы шлюза-агрегатора интернет-магазина предъявляются повышенные требования к его технической и архитектурной основе. Система должна корректно функционировать при пиковых нагрузках, возникающих в периоды распродаж и акций, и поддерживать одновременную обработку большого числа платёжных операций. Серверная часть должна разворачиваться на отказоустойчивой инфраструктуре, минимальные требования к которой включают: процессор архитектуры x86-64 с не менее чем 8 ядрами; объём оперативной памяти от 32 ГБ; высокопроизводительную дисковую подсистему на базе SSD для хранения транзакционных данных, логов и служебной информации. Для обеспечения высокой доступности критичных компонентов рекомендуется использование кластерных решений с автоматическим переключением при сбоях и резервированием каналов связи.

Требования к производительности и надежности были определены и структурированы при построении диаграммы Utility Tree (рисунок 1.5), отражающей связь нефункциональных характеристик с ключевыми бизнес-сценариями обработки платежей. В части производительности система должна обеспечивать время отклика для 95-го перцентиля не более 150 мс для операций авторизации и не более 200 мс для проведения платежа. Уровень доступности ядра системы должен составлять не менее 99.9% в месяц, а время переключения на резервные компоненты при сбое — не превышать 5 минут. Система должна поддерживать масштабирование для обработки не менее 500 запросов в секунду на входные API и обеспечивать горизонтальное расширение без простоев, сохраняя стабильность работы при росте нагрузки.

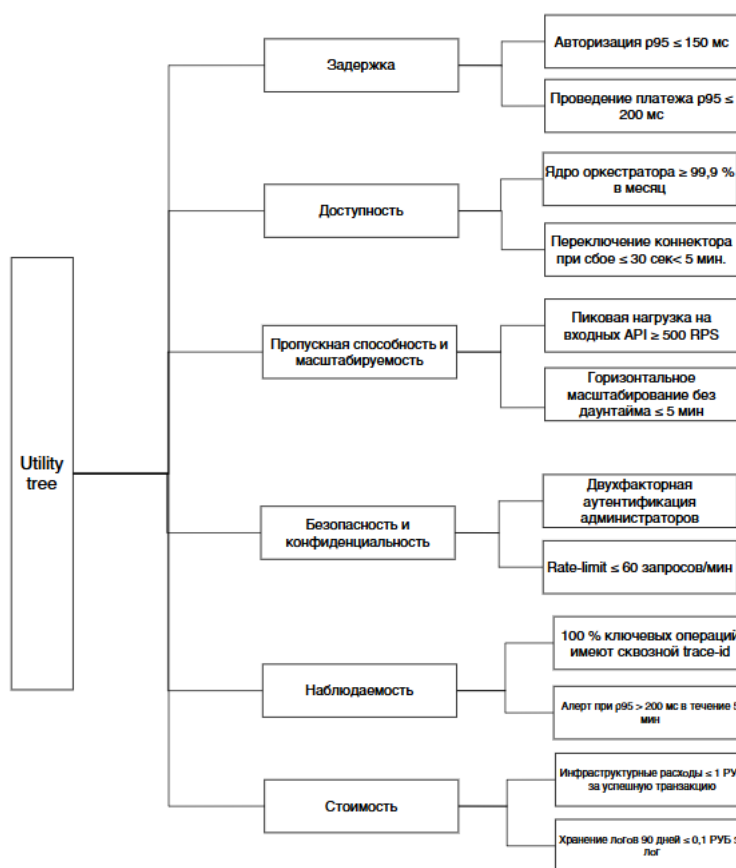


Рисунок 1.5 – Диаграмма Utility Tree

1.6.4 Требования к безопасности

Требования к безопасности системы включают обеспечение конфиденциальности, целостности и доступности платёжных данных, а также

строгое разграничение доступа в соответствии с ролями пользователей. Все передаваемые данные между клиентскими приложениями, системой мерчанта и платёжным шлюзом, а также взаимодействия с банковскими и внешними сервисами, должны быть защищены с использованием протокола TLS версии 1.3. Доступ к конфиденциальной информации должен осуществляться исключительно по защищённым соединениям.

В системе должна быть реализована многоуровневая ролевая модель доступа, разграничивающая права для ролей «Пользователь», «Бизнес-клиент» и «Администратор». Аутентификация должна выполняться с использованием безопасного механизма на основе JWT-токенов с обязательной валидацией на стороне сервера. Для доступа администраторов к критически важным функциям рекомендуется использовать многофакторную аутентификацию.

Все персональные и платёжные данные должны храниться в зашифрованном виде. Для защиты конфиденциальной информации должен применяться алгоритм AES-256. Секретные данные должны храниться в защищённом виде. Система должна обладать встроенной защитой от распространённых веб-уязвимостей, включая SQL-инъекции, XSS и CSRF. Публичные API должны быть оснащены механизмами ограничения скорости запросов для предотвращения атак на доступность. Критические операции, такие как обработка платежа или изменение его статуса, должны обладать свойством идемпотентности для обеспечения корректности обработки повторяющихся запросов.

1.6.5 Use-Case модель системы

Use-Case модель системы, представленная на рисунке 1.6, демонстрирует полный охват функциональности шлюза-агрегатора интернет-магазина и взаимодействие всех ключевых акторов. Модель включает основные сценарии: для пользователя — инициирование оплаты заказа, получение подтверждения и уведомлений о статусе платежа; для бизнес-клиента (мерчанта) — создание платёжного запроса и получение отчетов по

транзакциям; для администратора — управление тарифами и лимитами, а также регистрация и подключение новых мерчантов.

Также учтены взаимодействия с внешними системами, такими как банки и платежные системы для проведения списаний, антифрод-сервис для проверки транзакций на мошенничество и сервис уведомлений для информирования пользователей. Все ключевые прецеденты включают сценарии валидации и проверки транзакционных данных, а основные бизнес-события, такие как успешная оплата или изменение статуса платежа, расширяются отправкой уведомлений и логированием операций для последующего анализа.

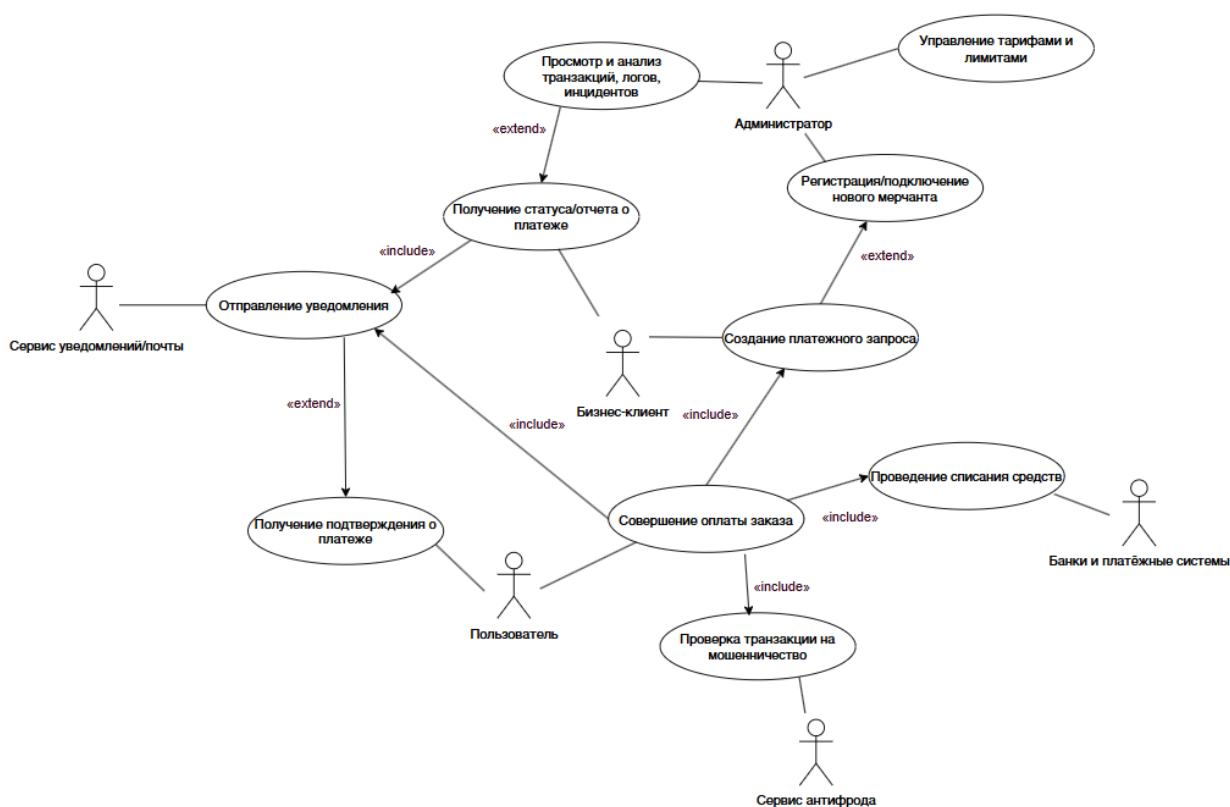


Рисунок 1.6 – Диаграмма Use-Case

1.6.6 Технические требования

Архитектура системы должна быть основана на микросервисном подходе с выделенными сервисами и единой точкой входа в виде API Gateway. Клиентская часть представлена интеграцией со стороны систем мерчантов, обеспечивающих взаимодействие пользователей с платёжным шлюзом. Для

взаимодействия с внешними системами должны использоваться стандартные REST API. Хранение структурированных данных осуществляется в реляционной СУБД PostgreSQL, для кэширования и обработки временных данных применяется Redis.

1.6.7 Бизнес-требования и ограничения

Ключевым бизнес-требованием является обеспечение надёжной, быстрой и безопасной обработки онлайн-платежей. Система должна способствовать сокращению времени обработки транзакций и повышению процента успешных платежей за счёт оптимизации маршрутизации и автоматизации проверки операций. Операционные затраты на обработку платежей и поддержку интеграций должны быть снижены за счёт централизованной архитектуры платёжного шлюза.

Ограничения проекта включают фиксированный бюджет на вычислительную инфраструктуру, способную обрабатывать большое количество параллельных транзакций, и срок разработки, не превышающий 9 месяцев. Система должна поддерживать стабильную работу при высокой нагрузке, обеспечивать отказоустойчивость и гарантировать защиту персональных и платёжных данных пользователей в соответствии с требованиями законодательства (например, ФЗ-152 «О персональных данных»).

1.7 Выводы и результаты к разделу

В первом разделе выполнено проектирование концептуальной модели системы платёжного шлюза-агрегатора. Проведён анализ предметной области, выявлены ключевые процессы обработки платежей и роли участников (пользователь, бизнес-клиент, администратор). На основе анализа существующих решений обоснована необходимость разработки системы, обеспечивающей централизованную обработку платежей, интеграцию с внешними сервисами и высокий уровень безопасности.

В качестве методологии концептуального моделирования выбрана нотация С4, позволившая описать систему на различных уровнях абстракции.

Разработаны диаграммы System Landscape, System Context и Container, которые продемонстрировали архитектуру системы и её взаимодействие с внешними сервисами.

Сформулированы функциональные и нефункциональные требования, включая требования к производительности, доступности, безопасности и масштабируемости системы. Определены ключевые технические решения, основанные на микросервисной архитектуре, а также установлены основные бизнес-ограничения проекта.

2 РАЗРАБОТКА ЛОГИЧЕСКОЙ МОДЕЛИ КЛИЕНТ-СЕРВЕРНОЙ СИСТЕМЫ

2.1 Выбор методологии и технологии логического моделирования клиент-серверной системы

Для логического моделирования клиент-серверной системы платёжного шлюза-агрегатора проводится выбор современных методологий объектно-ориентированного проектирования. На основе сравнительного анализа, выполненного в разделе 1.3, для детальной проработки логической архитектуры выбрана методология UML (Unified Modeling Language). Данный выбор обусловлен её способностью обеспечивать комплексное и непротиворечивое моделирование как статической структуры данных, так и динамического поведения компонентов в рамках ключевых сценариев обработки платежей, таких как инициация и выполнение транзакции.

Объектно-ориентированный подход, лежащий в основе UML, соответствует выбранной для реализации микросервисной архитектуре и технологическому стеку. Это обеспечивает плавный переход от логических моделей к физической реализации, поскольку диаграммы классов, последовательностей и состояний могут быть непосредственно отображены в структуры сущностей (например, «Платёж», «Транзакция», «Проверка антифрода»), схемы базы данных и контракты взаимодействия между специализированными сервисами.

Для полного охвата логической архитектуры системы в рамках данной работы разрабатывается комплекс взаимосвязанных UML-диаграмм. Диаграмма классов отображает структуру основных сущностей предметной области и отношения между ними. Диаграмма последовательностей детализирует взаимодействие участников и компонентов системы в ключевых сценариях, таких как «Обработка онлайн-платежа». Диаграмма состояний описывает жизненный цикл платёжной транзакции, фиксируя все возможные статусы и правила переходов между ними. Такой комплексный подход обеспечивает необходимую степень детализации и согласованности моделей,

создавая основу для последующего этапа физического проектирования и реализации системы.

2.2 Разработка диаграмм логической модели клиент-серверной системы

В данном разделе разработаны ключевые UML-диаграммы, отражающие логическую модель клиент-серверной системы оплаты онлайн-заказа. Диаграммы демонстрируют структуру данных, взаимодействие компонентов и логику ключевых бизнес-процессов на уровне предметной области.

Диаграмма классов на рисунке 2.1 отображает ключевые сущности системы и их взаимосвязи.

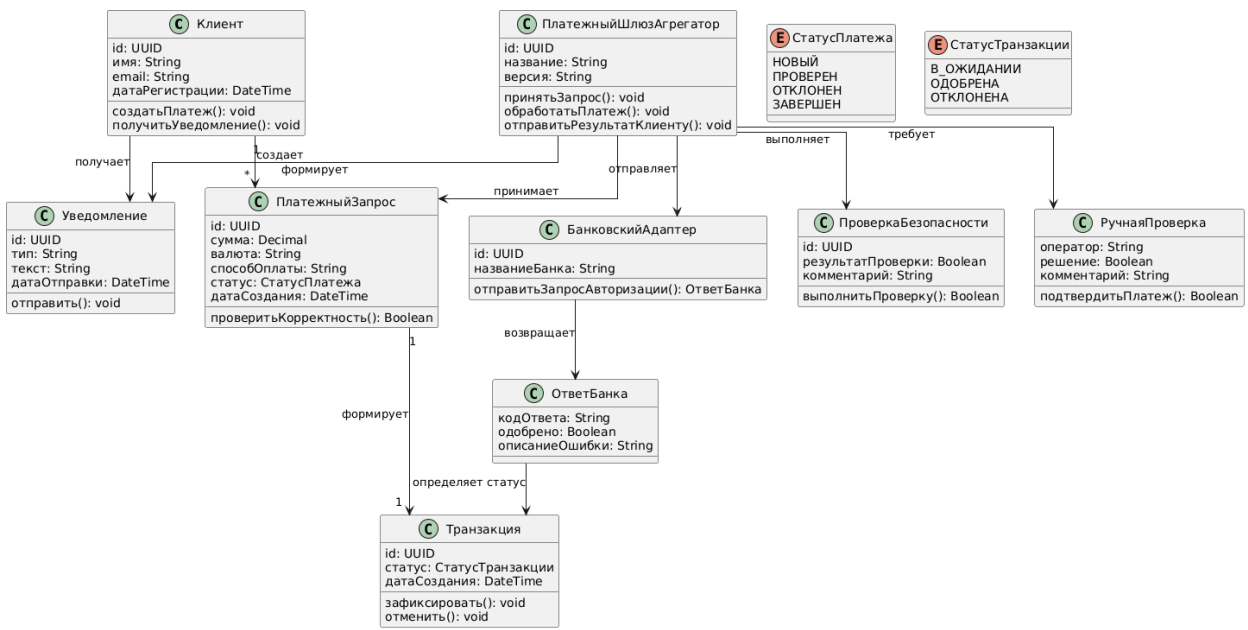


Рисунок 2.1 – Диаграмма классов

Центральное место в модели занимает класс ПлатежныйЗапрос, который инкапсулирует информацию о транзакции: сумму, валюту, способ оплаты и статус обработки. ПлатежныйЗапрос связан отношением ассоциации с классом Клиент, так как инициация и параметры платежа всегда задаются конкретным пользователем, а также с классом Транзакция, который представляет собой результат выполнения запроса в платёжной системе. Для управления процессом маршрутизации введён класс ПроверкаБезопасности,

находящийся в отношении композиции с классом ПлатежныйЗапрос, что отражает прямую зависимость этапа фрод-мониторинга от факта создания платежа. Классы Клиент, ПлатежныйЗапрос и ОтветБанка, хотя формально и не наследуются от общего предка, реализуют единую логику платёжного взаимодействия, разделяя общие атрибуты и специфическое поведение, характерное для каждой стадии жизненного цикла оплаты.

Диаграмма последовательности на рисунке 2.2 детально описывает процесс, выполняемый пользователем интернет-магазина — «Совершение платежа».

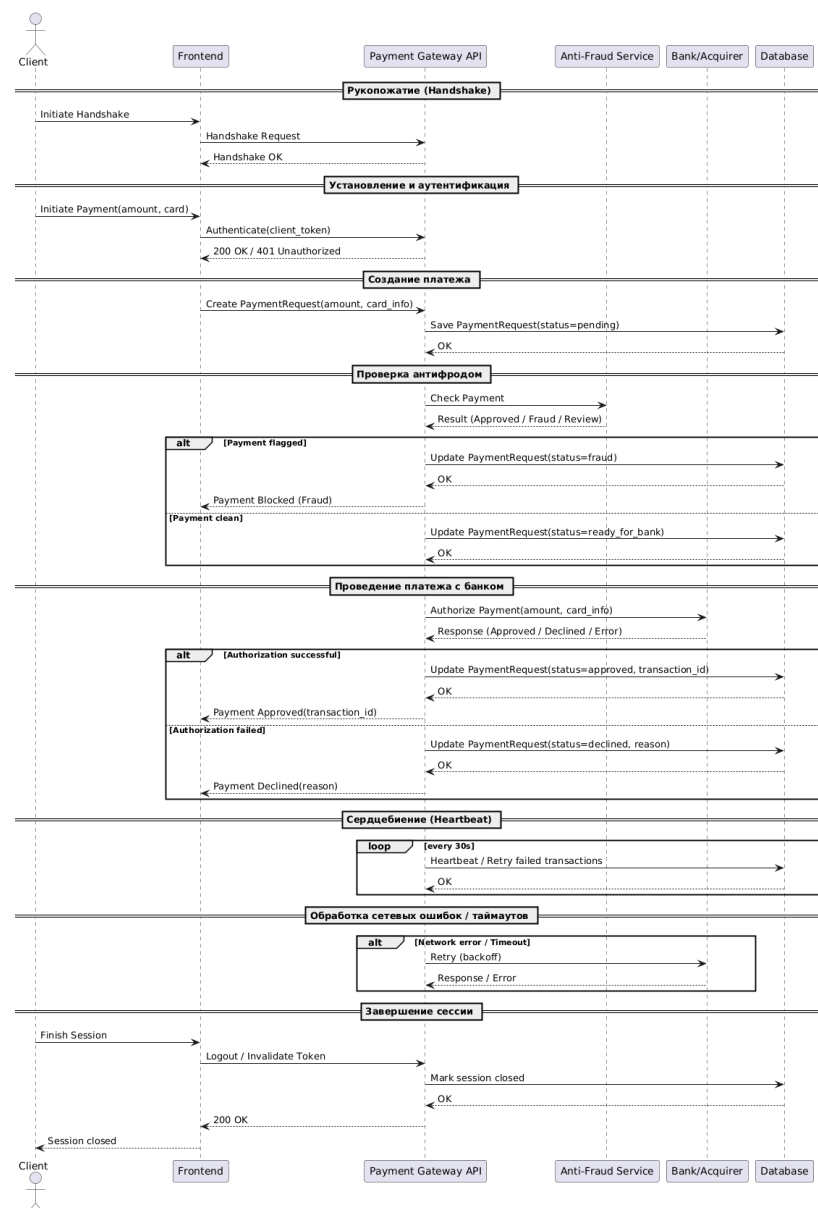


Рисунок 2.2 – Диаграмма последовательности

Диаграмма отражает сценарий «Обработка платежа с учетом ошибок и проверок». Процесс начинается с установления соединения (handshake) между клиентским приложением и шлюзом оплаты через frontend. После успешного установления соединения система выполняет аутентификацию клиента с использованием токена. В случае ошибки авторизации процесс прерывается с возвратом соответствующего статуса.

При успешной аутентификации инициируется создание платежного запроса, который сохраняется в базе данных со статусом ожидания. Далее система передает транзакцию в антифрод-сервис для проверки на мошенничество. В зависимости от результата проверка может либо заблокировать платеж, либо разрешить дальнейшую обработку.

Если транзакция признана безопасной, шлюз направляет запрос в банк или платежную систему для проведения авторизации списания средств. В зависимости от ответа банка платеж либо подтверждается с сохранением идентификатора транзакции, либо отклоняется с указанием причины. Все изменения статусов фиксируются в базе данных.

Дополнительно на диаграмме показаны механизмы повышения надежности: периодическая отправка heartbeat-сообщений для контроля состояния системы, повторные попытки выполнения операций при сетевых ошибках и таймаутах, а также корректное завершение пользовательской сессии с инвалидированием токена.

Диаграмма состояний на рисунке 2.3 и 2.4 демонстрирует полный жизненный цикл процесса оплаты.

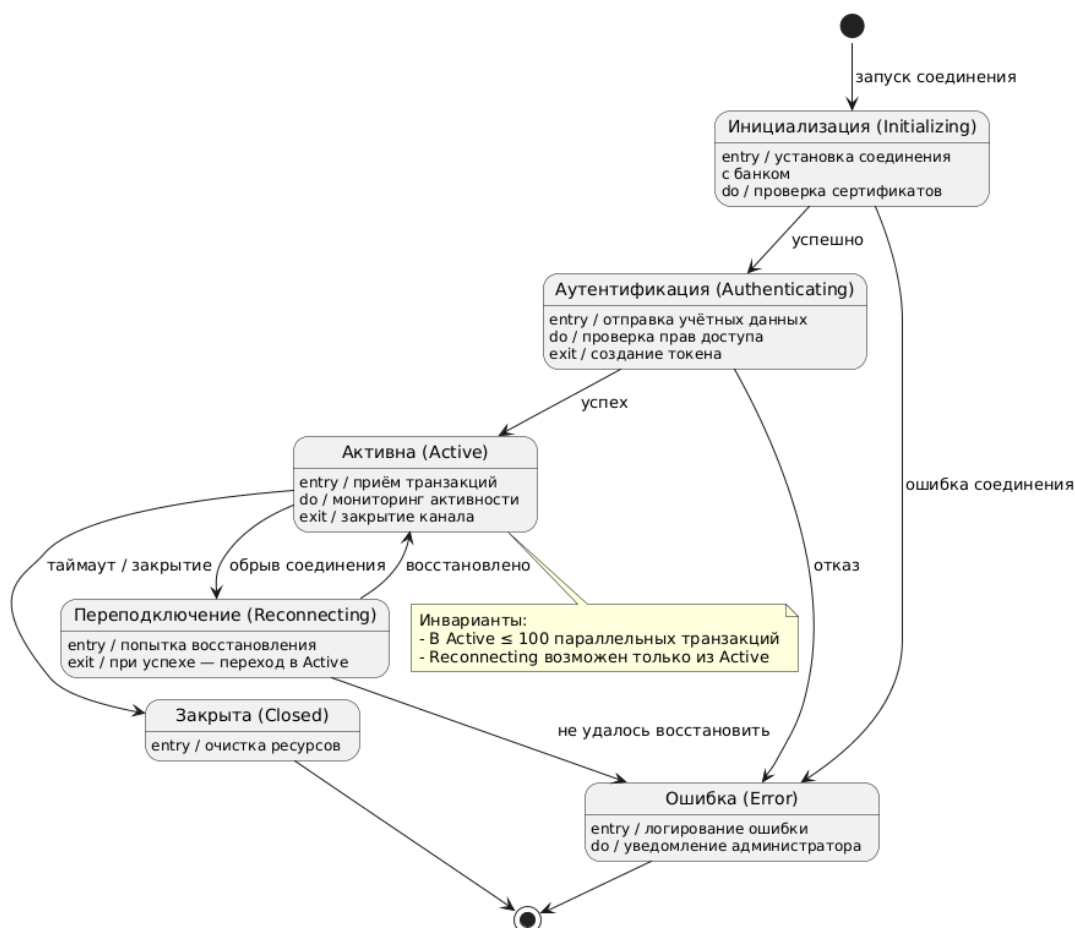


Рисунок 2.3 – Диаграмма состояний

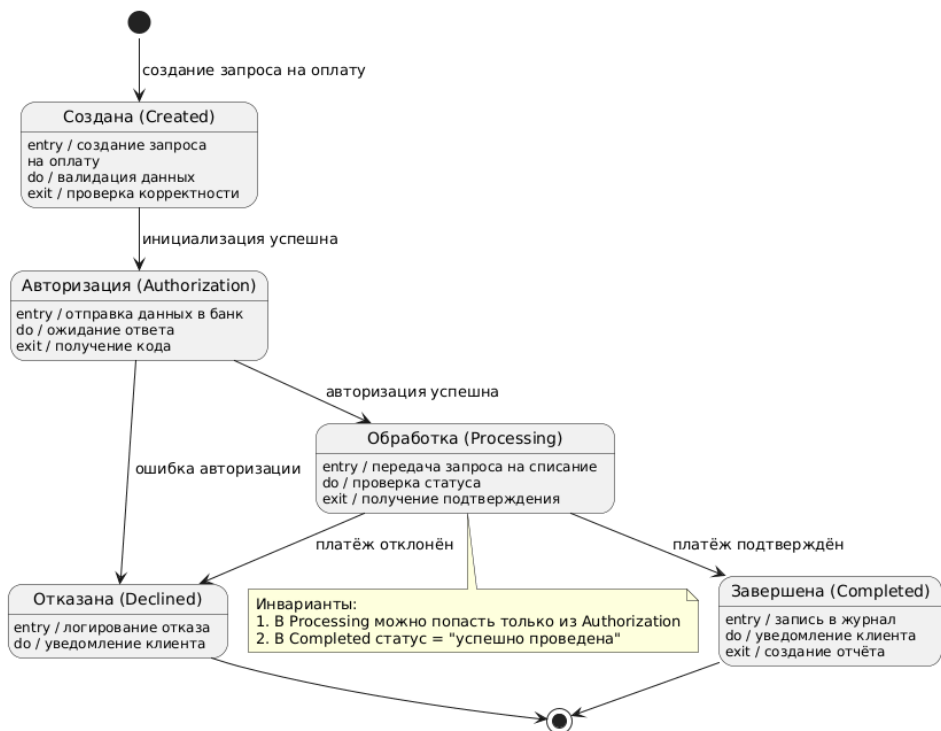


Рисунок 2.4 – Диаграмма состояний

Диаграмма состояний описывает жизненный цикл соединения с банком-эквайером в системе платёжного агрегатора. Начальным состоянием является *Initializing*, когда шлюз устанавливает сетевое подключение к платёжной системе банка. После запуска происходит переход в состояние *Authenticating*, где отправляются учётные данные мерчанта (терминала) и проверяются права доступа, а при успешной аутентификации генерируется токен сессии.

Далее шлюз переходит в состояние *Active*, в котором он способен принимать и обрабатывать платёжные транзакции. В рамках этого состояния ведётся мониторинг активности канала и соблюдается инвариант: не более 100 параллельных транзакций. При возникновении таймаута или обрыва соединения шлюз переходит в состояние *Reconnecting*, где система пытается восстановить канал. Если попытки оказываются успешными, сессия возобновляется и возвращается в *Active*.

При критической ошибке соединение переходит в состояние *Error*, где происходит логирование инцидента и уведомление администратора. Если восстановление после многократных попыток не удалось, или шлюз штатно завершает работу, соединение переходит в состояние *Closed*, где выполняются очистка ресурсов и закрытие каналов.

Вторая диаграмма описывает жизненный цикл отдельного платёжного запроса. Начальным состоянием является *Created*, где запрос создаётся, валидируются данные (сумма, валюта, реквизиты) и проверяется корректность. После успешной инициализации запрос переходит в состояние *Authorization*, где отправляются данные в банк, ожидается ответ и получается код авторизации.

При успешной авторизации запрос переходит в состояние *Processing*, где выполняется передача запроса на списание средств и проверка статуса. Из этого состояния возможны два исхода: подтверждение платежа (переход в *Completed*) или отклонение (переход в *Declined*). В состоянии *Completed* производится запись в журнал, уведомление клиента и

создание отчёта, а в Declined — логирование отказа и уведомление клиента об отклонении. Инвариантами являются: переход в Processing только из Authorization, а также то, что в состоянии Completed статус платежа считается успешно проведённым.

2.3 Разработка логической модели данных клиент-серверной системы

На основе проанализированных бизнес-процессов и разработанных диаграмм логической модели была спроектирована логическая модель данных системы платёжного шлюза-агрегатора в нотации ER, изображённая на рисунке 2.5.

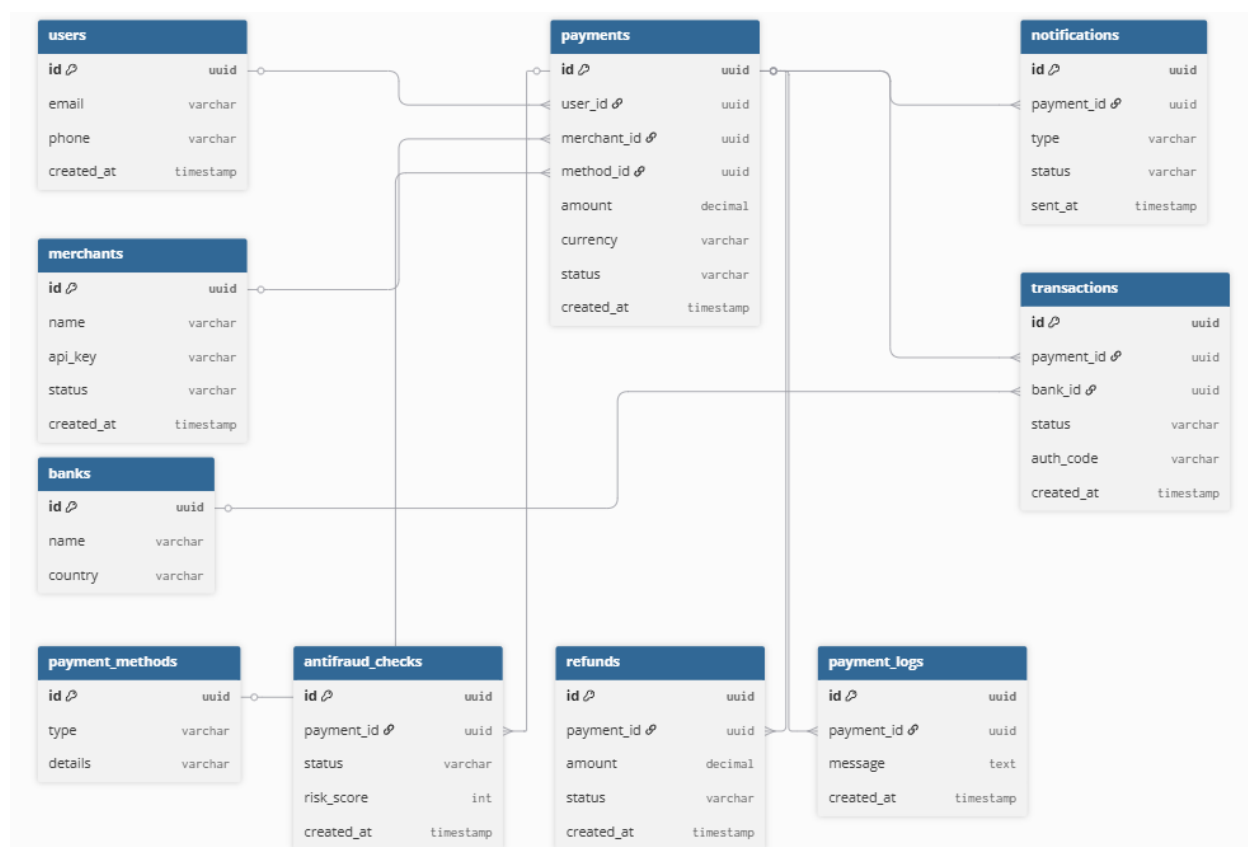


Рисунок 2.5 – ER диаграмма

Центральной сущностью модели является payments, которая хранит данные о проводимых транзакциях и их параметрах (сумма, валюта, статус). Она связана с сущностью users, отвечающей за инициацию платежей, и множеством сущностей, описывающих процесс обработки. Основными

участниками системы являются users, merchants и banks, каждый из которых взаимодействует с платежами через специализированные сущности.

Процесс проведения платежа моделируется через связку сущностей transactions, antifraud_checks и refunds. Каждый активный платёж сопровождается логированием в сущности payment_logs, где фиксируются все этапы обработки и потенциальные ошибки. Итогом успешного проведения платежа является генерация записи в transactions с подтверждённым статусом и авторизационным кодом банка. Информирование пользователей о статусе оплаты осуществляется через сущность notifications. Способы оплаты (payment_methods) и подключённые банки (banks) настраиваются независимо и связываются с платежами через соответствующие внешние ключи.

Модель построена с использованием UUID для уникальных идентификаторов всех записей. Все таблицы содержат метки времени создания (created_at) для аудита, а ключевые сущности также включают поле обновления. Такая структура обеспечивает целостность данных, эффективность выполнения запросов и полностью соответствует логике проведения безопасных и отслеживаемых платежей через агрегирующий шлюз.

2.4 Разработка архитектуры клиент-серверной системы с использованием паттерна, целесообразность которого обоснована

Архитектурное решение системы платёжного шлюза-агрегатора разработано в нотации ArchiMate (рисунок 2.6).

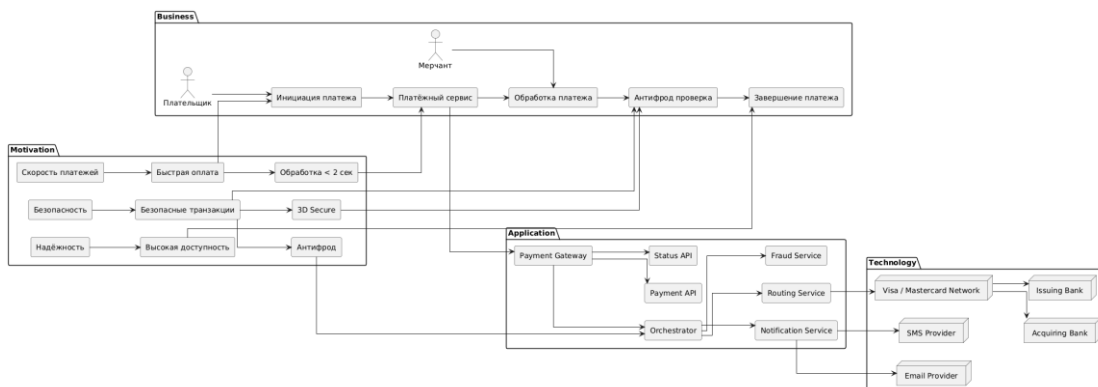


Рисунок 2.6 – Диаграмма ArchiMate

Архитектурное решение системы агрегации онлайн-платежей разработано в нотации ArchiMate и охватывает пять ключевых уровней. Мотивационный уровень определяет стратегические цели и требования, включая ускорение процесса обработки платежей, обеспечение безопасности транзакций и выполнение нефункциональных требований по производительности (обработка ≤ 2 секунд) и надёжности (высокая доступность, антифрод). Эти цели поддерживаются ключевой стратегической способностью — цифровизацией платёжного процесса.

На бизнес-уровне выделены три ключевых процесса: «Инициация платежа», «Обработка платежа» и «Антифрод проверка». Каждый процесс обслуживается специализированными бизнес-услугами, такими как «Платежный сервис» для проведения транзакций и «Сервис завершения платежа» для фиксации результатов, что обеспечивает сквозную автоматизацию платёжного цикла.

Уровень приложений реализует микросервисную архитектуру с единой точкой входа в виде Payment API, что обеспечивает централизованную аутентификацию и маршрутизацию запросов. Для оптимизации взаимодействия с разными категориями участников (плательщики, мерчанты) используются выделенные сервисы — Payment Gateway, Routing Service, Fraud Service, Status API и Notification Service, которые агрегируют данные от основных компонентов, включая оркестратор процессов. Выбор данного подхода обоснован необходимостью независимого масштабирования ресурсоёмких компонентов (например, антифрод проверки и маршрутизации) и упрощения поддержки разнородных клиентских приложений.

Технологический уровень описывает инфраструктурную реализацию. Плательщик взаимодействует с платёжным шлюзом через сети Visa/Mastercard. Серверные компоненты развёрнуты в виде контейнеризированных микросервисов, взаимодействующих с реляционной базой данных для хранения структурированных данных о платежах, а также с

внешними провайдерами — банком-эмитентом, банком-эквайером, SMS-провайдером и Email-провайдером для уведомлений.

Таким образом, выбранная архитектура на основе оркестрации микросервисов и специализированных платёжных сервисов обеспечивает достижение всех стратегических целей, предоставляя необходимую производительность, безопасность (3D Secure), масштабируемость и удобство поддержки.

2.5 Выводы и результаты к разделу

Во втором разделе выполнена разработка логической модели шлюза-агрегатора онлайн-платежей. В качестве основной методологии логического моделирования выбрана UML, что позволило комплексно охватить статическую структуру данных и динамическое поведение системы в рамках ключевых платёжных сценариев.

Были разработаны ключевые UML-диаграммы: диаграмма классов, отображающая основные сущности предметной области и их взаимосвязи; диаграмма последовательности, детально описывающая процесс обработки платежа от инициации до получения ответа банка; и диаграмма состояний, демонстрирующая полный жизненный цикл соединения с банком-эквайером и отдельного платёжного запроса.

На основе этих диаграмм была создана логическая модель данных в нотации ER, транзакций, способов оплаты, антифрод-проверок, возвратов и уведомлений. Разработанные модели служат согласованной и детальной основой для последующего проектирования физической архитектуры и непосредственной реализации системы.

Архитектура системы была спроектирована с применением нотации ArchiMate и основана на микросервисном подходе с выделением специализированных сервисов. Данное архитектурное решение обеспечивает высокую масштабируемость, отказоустойчивость и способствует достижению ключевых стратегических целей проекта по цифровизации и обеспечению безопасности платёжных транзакций в режиме реального времени.

ЗАКЛЮЧЕНИЕ

В ходе проектирования шлюза-агрегатора онлайн-платежей был проведён комплексный анализ предметной области и современных подходов к разработке. На основании исследования существующих платёжных процессов, оценки требований и сравнительного анализа готовых решений была выбрана микросервисная архитектура с оркестрацией сервисов. Такой подход доказал свою целесообразность, позволив учесть специфику проведения безопасных онлайн-транзакций, обеспечить необходимую масштабируемость, гибкость интеграций с внешними банковскими системами и высокую производительность при обработке платежей в реальном времени.

В результате работы были достигнуты все поставленные цели и задачи: разработана концептуальная модель системы, включая анализ бизнес-процессов и проектирование целевой модели; сформулированы детальные функциональные и нефункциональные требования (скорость обработки ≤ 2 секунд, высокая доступность, антифрод-защита). Итогом проектирования стала комплексная логическая модель системы, представленная набором UML-диаграмм (классов, последовательности, состояний) и ER-моделью данных, а также архитектурная модель в нотации ArchiMate. Все модели взаимосвязаны и служат надёжным основанием для последующей физической реализации.

Спроектированная система полностью покрывает ключевые процессы проведения онлайн-платежей: обеспечивает безопасный и удобный интерфейс для инициации платежей плательщикам, инструменты для приёма и обработки транзакций мерчантам, а также возможности антифрод-мониторинга, маршрутизации через разные банки и автоматического уведомления о статусах. Реализация предложенного архитектурного решения позволит существенно повысить скорость обработки платежей, автоматизировать рутинные операции, минимизировать риски мошенничества и повысить доверие к цифровым расчётам, что в совокупности укрепит эффективность и надёжность платёжной инфраструктуры.